B.TECH 1ST YEAR · 1ST SEMESTER · UNIT I

Data Structures

Linear DS · Searching · Sorting — Complete Revision Sheet

Level	Pages	Unit
B.Tech	10–12	Unit I

Linear DS · ADTs · Complexity · Linear Search · Binary Search · Bubble Sort · Selection Sort · Insertion Sort · Flashcards · Exam Tips

1 Linear Data Structures

FOUNDATION

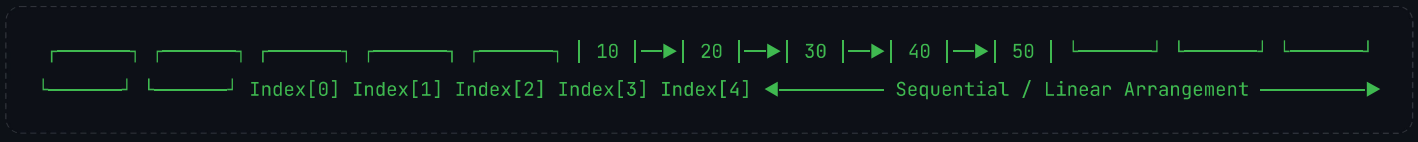
DEFINITION

A **linear data structure** organises data elements in a *sequential order* where each element has exactly one predecessor and one successor (except the first and last).

- Elements stored **contiguously or linked**
- Single traversal path (one direction)
- Easy to implement & traverse

EXAMPLES

STRUCTURE	REAL-WORLD
Array	Marks list
Linked List	Music playlist
Stack	Undo / Redo
Queue	Printer queue



2 Abstract Data Types (ADTs)

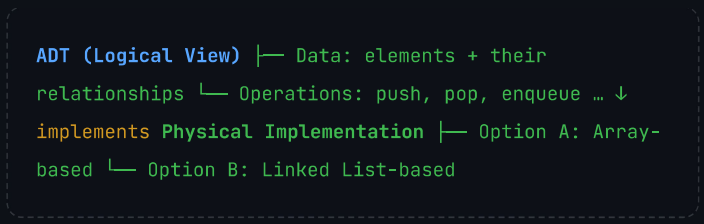
ABSTRACTION

DEFINITION

An **ADT** is a mathematical model describing a data type by its *behaviour* (operations) from the user's point of view, independent of implementation.

ASPECT	LOGICAL (ADT)	PHYSICAL (IMPL.)
View	What it does	How it's done
User sees	Operations	Array / LL

Data + Operations = ADT
Stack ADT → `push()`, `pop()`, `peek()`, `isEmpty()`



ADVANTAGES OF ADTS

- **Abstraction** – hide implementation details
- **Reusability** – use same ADT everywhere
- **Flexibility** – swap implementation
- **Modularity** – clean software design

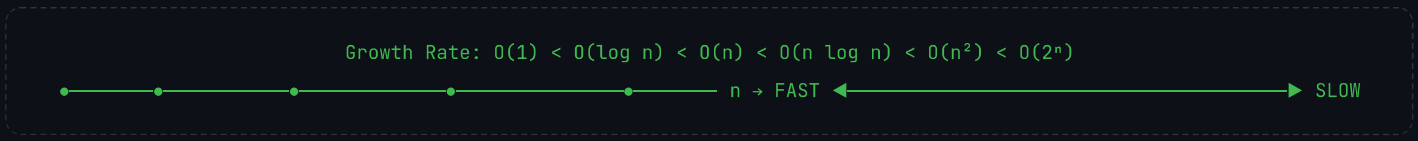
3 Time & Space Complexity

DEFINITIONS

- **Time Complexity** – number of operations as a function of input size *n*
- **Space Complexity** – extra memory used by algorithm
- **Big-O** – worst-case upper bound
- **Big-Ω** – best-case lower bound
- **Big-Θ** – tight / average bound

BIG-O HIERARCHY (FASTEST → SLOWEST)

NOTATION	NAME	EXAMPLE
O(1)	Constant	Array access
O(log n)	Logarithmic	Binary Search
O(n)	Linear	Linear Search
O(n log n)	Linearithmic	Merge Sort
O(n²)	Quadratic	Bubble Sort
O(2ⁿ)	Exponential	Recursion



ARRAY OPERATIONS – COMPLEXITY

OPERATION	ARRAY	LINKED LIST
Access by index	O(1)	O(n)
Search (unsorted)	O(n)	O(n)
Insert (beginning)	O(n)	O(1)
Insert (end)	O(1)	O(1)
Delete	O(n)	O(1)

BEST / WORST / AVERAGE CASES

CASE	MEANING	NOTATION
Best	Most favourable input	Ω (Omega)
Average	Expected input	Θ (Theta)
Worst	Least favourable input	O (Big-O)

> ⚠ Exams usually ask for **Worst-Case (Big-O).** Always mention it!

4 Searching Techniques

LINEAR & BINARY

🔍 LINEAR SEARCH

- Compare target with **each element** left → right
- Works on **unsorted** arrays
- Simple, no preprocessing needed

Best: O(1) | Avg: O(n/2) | Worst: O(n)

🔎 BINARY SEARCH

- Array **must be sorted**
- Divide search space in **half** each step
- Divide and Conquer approach

Best: O(1) | Avg: O(log n) | Worst: O(log n)

```
// LINEAR SEARCH
function linearSearch(arr, n, key):
  for i = 0 to n-1:
    if arr[i] == key:
      return i    // Found! Return index
  return -1       // Not found

// EXAMPLE: arr=[5,3,8,1], key=8
// Step 1: 5≠8 → Step 2: 3≠8 → Step 3: 8=8 ✓ return 2
```

```
// BINARY SEARCH
function binarySearch(arr, low, high, key):
  while low ⩽ high:
    mid = (low + high) / 2
    if arr[mid] == key:
      return mid         // Found!
    else if arr[mid] < key:
      low = mid + 1     // Search right half
    else:
      high = mid - 1    // Search left half
  return -1
```

```
arr = [ 2, 5, 8, 12, 16, 23, 38, 56 ] → key = 23 ↑ ↑ low=0 high=7 mid = (0+7)/2 = 3 → arr[3]=12 < 23 → low=4 ↑ ↑ low=4
                                high=7 mid = (4+7)/2 = 5 → arr[5]=23 = 23 ✓ FOUND at index 5
```

⚖️ LINEAR VS BINARY SEARCH — COMPARISON (EXAM MUST)

FEATURE	LINEAR SEARCH	BINARY SEARCH
Data requirement	Unsorted / Sorted	Sorted only
Time Complexity	O(n) worst	O(log n) worst
Best case	O(1)	O(1)
Space Complexity	O(1)	O(1)
Approach	Sequential scan	Divide & Conquer
Implementation	Simple	Moderate
Preferred when	Small / unsorted data	Large sorted data

5 Sorting Techniques

BUBBLE · SELECTION · INSERTION

BUBBLE SORT

- Compare **adjacent elements**, swap if out of order
- Largest element *bubbles up* to end each pass
- **n-1 passes** for n elements

Best: O(n) | Worst: O(n²) | Stable: ✓

```
// BUBBLE SORT
for i = 0 to n-2:
  swapped = false
  for j = 0 to n-i-2:
    if arr[j] > arr[j+1]:
      swap(arr[j], arr[j+1])
      swapped = true
  if not swapped: break  // Optimised
```

Pass-wise Example: arr = [5, 3, 8, 1]
Pass 1: [3,5,1,8] Pass 2: [3,1,5,8] Pass 3: [1,3,5,8] ✓

SELECTION SORT

- Find **minimum element** in unsorted portion
- Swap it to the correct position
- Always O(n²) — no early termination

Best: O(n²) | Worst: O(n²) | Stable: ✗

```
// SELECTION SORT
for i = 0 to n-2:
  minIdx = i
  for j = i+1 to n-1:
    if arr[j] < arr[minIdx]:
      minIdx = j
  swap(arr[i], arr[minIdx])
```

Step-wise Example: arr = [64, 25, 12, 22]
Pass 1: min=12 → [12, 25, 64, 22]
Pass 2: min=22 → [12, 22, 64, 25]
Pass 3: min=25 → [12, 22, 25, 64] ✓

↗ 🔁 Insertion Sort

- Build sorted portion **one element at a time**
- Pick each element and **insert** into correct position in sorted half
- Best case when array is nearly sorted

Best: O(n) | Worst: O(n²) | Stable: ✓

```
// INSERTION SORT
for i = 1 to n-1:
  key = arr[i]
  j = i - 1
  while j ≥ 0 and arr[j] > key:
    arr[j+1] = arr[j]  // Shift right
    j = j - 1
  arr[j+1] = key      // Place key
```

```
arr = [12, 11, 13, 5, 6] Step 1: key=11 [11, 12, 13, 5, 6] Step 2: key=13 [11, 12, 13, 5, 6] Step 3: key=5 [5, 11, 12, 13, 6] Step 4: key=6 [5, 6, 11, 12, 13] ✓ Sorted
◀——| Key |—— Unsorted —▶
```

6 Sorting Algorithms — Master Comparison Table

⭐ EXAM MUST-KNOW

ALGORITHM	BEST CASE	AVERAGE CASE	WORST CASE	SPACE	STABLE	IN-PLACE	KEY IDEA
Bubble Sort	O(n)*	O(n²)	O(n²)	O(1)	Yes ✓	Yes ✓	Adjacent swap
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No ✗	Yes ✓	Select minimum
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes ✓	Yes ✓	Insert in sorted
Merge Sort †	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes ✓	No ✗	Divide & Merge
Quick Sort †	O(n log n)	O(n log n)	O(n²)	O(log n)	No ✗	Yes ✓	Pivot partitioning

* O(n) best for Bubble Sort only when optimised with early-exit flag. † For reference; may be in syllabus.

💡 **Stability**: A stable sort preserves the relative order of *equal* elements.
Bubble ✓ | Insertion ✓ | Selection ✗

💡 **In-Place**: Uses O(1) extra space (no auxiliary array). All three basic sorts are in-place.

7 Key Terms & Definitions

CORE CONCEPTS (A–L)

TERM	DEFINITION
Algorithm	Finite, ordered steps to solve a problem
ADT	Logical model of data + its operations
Array	Fixed-size collection of same-type elements
Big-O	Upper bound of time/space complexity
Best Case	Minimum operations for any input
Binary Search	Search by halving sorted array
Bubble Sort	Sort by repeatedly swapping adjacent elements
Complexity	Measure of algorithm efficiency
Data Structure	Method of organising and storing data
Divide & Conquer	Split problem, solve, combine results
In-Place	Sort using O(1) extra memory
Insertion Sort	Build sorted array one element at a time
Iteration	Repeated execution of a loop block
Key	Target value being searched for
Linear Search	Scan each element sequentially
Linear DS	Sequential arrangement of elements
Linked List	Dynamic nodes linked via pointers

CORE CONCEPTS (M–Z)

TERM	DEFINITION
Mid Index	(low + high) / 2 in binary search
Pass	One complete traversal in a sort algorithm
Pseudocode	Informal algorithm description in plain language
Queue	FIFO linear data structure
Recursion	Function calling itself
Searching	Finding a target element in a dataset
Selection Sort	Repeatedly select minimum and place it
Sorted Array	Elements arranged in ascending/descending order
Sorting	Arranging data in a specific order
Space Complexity	Extra memory used by an algorithm
Stable Sort	Maintains order of equal elements
Stack	LIFO linear data structure
Swap	Exchange values of two variables
Time Complexity	Operations count relative to input size
Traversal	Visiting every element of a structure
Unsorted Array	No particular order of elements
Worst Case	Maximum operations for any input

8 Formula & Syntax Reference Box

MUST MEMORISE

Binary Search Mid Formula:

```
mid = (low + high) / 2
```

No. of passes (Bubble Sort):

```
passes = n - 1
```

No. of comparisons (Selection Sort):

```
comparisons = n(n-1) / 2
```

Binary Search — Max iterations:

$$\text{iterations} = \lfloor \log_2(n) \rfloor + 1$$

Linear Search worst case:

```
comparisons = n (element not found)
```

Swap Template:

```
temp = arr[i]
arr[i] = arr[j]
arr[j] = temp
```

Insertion Sort — Shift:

```
arr[j+1] = arr[j] // while arr[j] > key
```

Bubble inner loop bound:

```
j from 0 to (n - i - 2)
```

Selection outer loop:

```
i from 0 to (n - 2)
```

9 Problem-Solving Patterns

STRATEGY

🔍 WHEN TO USE WHICH SEARCH?

- Array **unsorted** → Linear Search
- Array **sorted** → Binary Search
- Small array (n < 10) → Either works
- Large array, sorted → Always Binary
- Linked List → Only Linear (no random access)

📊 WHEN TO USE WHICH SORT?

- **Nearly sorted** → Insertion Sort
- **Small n** → Any $O(n^2)$ sort
- **Min swaps needed** → Selection Sort
- **Teaching/Simple** → Bubble Sort
- **Large n** → Merge / Quick Sort

🧠 DIVIDE & CONQUER

- **Divide**: Split problem into subproblems
- **Conquer**: Solve each subproblem
- **Combine**: Merge solutions
- Used in: Binary Search, Merge Sort
- Reduces O(n) to O(log n)

Pattern Recognition: If a question gives a *sorted array* and asks to find an element efficiently — always write Binary Search. If the array is unsorted — use Linear Search or sort first then Binary Search.

10 Flashcards — Quick-Fire Revision

30 CARDS

Q: What is a linear data structure?
A: Data stored in sequential order, one element after another

Q: What is an ADT?
A: A logical model defining data and its operations, independent of implementation

Q: Binary search requirement?
A: Array must be sorted

Q: Best case of linear search?
A: O(1) — element found at first position

Q: Worst case of binary search?
A: O(log n)

Q: Best case of insertion sort?
A: O(n) — array already sorted

Q: Worst case of bubble sort?
A: $O(n^2)$ — reverse sorted array

Q: Is selection sort stable?
A: No — it is not stable

Q: Is insertion sort stable?
A: Yes — it preserves relative order

Q: What does O(1) mean?
A: Constant time — independent of input size

Q: Mid index formula in binary search?
A: mid = (low + high) / 2

Q: How many passes does bubble sort make?
A: n-1 passes for n elements

Q: What is selection sort's complexity?
A: $O(n^2)$ for best, average, and worst cases

Q: Stack follows which principle?
A: LIFO — Last In, First Out

Q: Queue follows which principle?
A: FIFO — First In, First Out

Q: What is an in-place sort?
A: Sort that uses O(1) extra memory space

Q: What does Big-O notation represent?
A: Worst-case upper bound of complexity

Q: Which search uses divide and conquer?
A: Binary Search

Q: Key idea of bubble sort?
A: Repeatedly swap adjacent elements if out of order

Q: Key idea of selection sort?
A: Find minimum, place it at correct position

Q: Key idea of insertion sort?
A: Pick element, insert into sorted portion

Q: Complexity of accessing arr[i]?
A: O(1) — direct index access

Q: Space complexity of basic sorts?
A: O(1) — all three are in-place

Q: What is O(log n)?
A: Logarithmic — input halved each step

Q: Fastest growing complexity class among these?
A: $O(n^2)$ — quadratic growth

Q: Which sort is best for nearly sorted arrays?
A: Insertion Sort — O(n) best case

Q: Insertion sort inner loop does what?
A: Shifts elements right to make room for key

Q: Linear search on linked list complexity?
A: O(n) — no random access possible

Q: Stable sort means?
A: Equal elements maintain original relative order

Q: What does traversal mean?
A: Visiting every element of a data structure once

11 Exam Tips & Common Mistakes

SCORE BOOSTER

❌ COMMON MISTAKES TO AVOID

- Forgetting that Binary Search **requires sorted array** — always state this condition
- Wrong loop bounds in Bubble Sort — inner loop should go to `n-i-2`
- Confusing `mid = (low+high)/2` — integer division, not float
- Saying Selection Sort is stable — it is **not stable**
- Writing O(n) for Selection Sort best case — it is always $O(n^2)$
- Not initialising `minIdx = i` inside outer loop of Selection Sort
- Forgetting `swapped` flag optimisation in Bubble Sort
- Writing insertion sort outer loop from `i=0` — should start from `i=1`
- Not returning -1 when element is not found in search algorithms

✅ WRITING TIPS FOR FULL MARKS

- Always **state precondition** for Binary Search (array must be sorted)
- Write **pseudocode step-by-step** with clear indentation
- Always **mention Best, Average & Worst** complexity separately
- **Draw step-by-step trace** for sorting — examiners love it
- Mention **Stable / In-Place** properties in sorting answers
- For ADTs — always say "logical view" vs "physical implementation"
- Use **diagrams for Binary Search** — show low, mid, high arrows
- In comparisons, always use a **table format** — clear marks
- Write the **swap operation explicitly**: temp → arr[i] → arr[j] → temp

🎯 **5-Mark Answer Formula:**
Definition (1) + Algorithm/Pseudocode (2) + Example with steps (1) + Complexity (1)

🎯 **10-Mark Answer Formula:**
Definition + Conditions + Pseudocode + Step-by-step trace + Complexity table + Diagram + Comparison

12 Quick Revision Sheet

LAST-DAY CHEAT SHEET

🔍 SEARCHING — COMPLEXITIES

LINEAR BEST	O(1)
LINEAR WORST	O(n)
BINARY BEST	O(1)
BINARY WORST	O(log n)
BINARY CONDITION	SORTED ARRAY

🔁 SORTING — COMPLEXITIES

BUBBLE BEST	O(n)
BUBBLE WORST	$O(n^2)$
SELECTION BEST	$O(n^2)$
SELECTION WORST	$O(n^2)$
INSERTION BEST	O(n)
INSERTION WORST	$O(n^2)$

⚡ KEY FORMULAS

```
mid = (low + high) / 2
Bubble passes = n - 1
Selection comparisons = n(n-1)/2
Max binary iterations = log₂(n) + 1
```

⚖️ KEY DIFFERENCES

PROPERTY	BUBBLE	SELECT	INSERT
Best Case	O(n)	$O(n^2)$	O(n)
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Stable	Yes	No	Yes
Swaps	Many	Few	Medium

🧠 ADT SUMMARY

- Stack → push, pop, peek, isEmpty
- Queue → enqueue, dequeue, front
- ADT = Data + Operations
- Implementation: Array or Linked List

📈 COMPLEXITY ORDER

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$$

← FAST SLOW →

⭐ Last-Minute Recalls:

- Binary Search = Sorted array + mid = (l+h)/2
- Selection Sort = NOT stable
- Insertion Sort best = O(n) [nearly sorted]
- All 3 basic sorts: In-place, O(1) space